

Program Design:

The program is coded in Python 3 and consists of two distinct Python files: "server.py" and "client.py," each residing in separate directories. The program initiates by launching the "server.py" script, allowing users to simultaneously open multiple instances of "client.py" with distinct logins.

Through the server, clients can perform a range of functions, such as sending private messages to other clients using the "/msgto" command, checking the list of active users with "/activeuser," creating group chats with "/creategroup," joining existing groups via "/joininggroup," broadcasting messages to all members of a group using "/groupmsg," and logging out with "/logout." Additionally, clients have the capability to exchange files through peer-to-peer connections with the "/p2pvideo" feature. When a client logs out, it becomes a deactivated client, but the message his/her sends will still be recorded in the userlog.txt and GROUPNAME_messageLog.txt. Empty messages will be considered invalid.

Text File

credential.txt: Valid usernames and passwords are initially read from "credential.txt" upon server startup and stored within the program for user authentication. Any new entries in "credential.txt" after program launch will not be recognised as valid logins.

messagelog.txt : Record all private message

userlog.txt : Record all the active user at the time

GROUPNAME_messageLog.txt : record all the message send in GROUPNAME

Initial Phrase

Server:

The server initializes two empty files, 'userlog.txt' and 'messagelog.txt,' and retrieves valid login credentials (username and password) as global variables in the validUser dictionary. It listens on the IP address obtained via the hostname, gets the corresponding IP using gethostbyname(hostname), and operates on the user-specified port number (provided as the first argument).

Additionally, the server enforces a 10-second login block for a user who experiences consecutive login failures (provided as the second argument) with a valid username (i.e a valid username but an incorrect password). Invalid usernames do not trigger this blocking mechanism.

Global dict and list in the server

validUser : validUser[username] = password

failDict : record the number of fail for each valid user and user that have success login will remove from the failDict; failDict[username] = Number of fail

blockUser: key is the store the user that is blocked, value is the end of their blocking

activeUser: activeUser[username] = the clientSocket of that client

groupDict : the key is a group name, the value is a dictionary where the key is the member of the group and the value of this sub-dictionary is whether the member has join the group or not (True/False)

Client:

Each client creates a TCP connection with the server and creates a UDP server themselves for downloading files from their peers.

global Variable in the client.py:

activeUserList: store all the activeUser at the time for /p2pvideo

clientActive: Whether the client is active or not (True / False)

Application Layer Message Format:

The client and server communicate through sending and dictionary objects.

The client request message looks like:

Sending request for login

```
request = {
    "username" : username,
    "password" : password
}
```

Sending request such as /msgto, /activeuser, /creategroup

```
request = {
    "command": message.split()[0]
}
```

The response from server to the client is design in similar to HTTP response which contain the header, statusCode and the returnMsg (i.e payload) i.e

```
response = {
    "header": httpRequest["command"],
    "statusCode" : "200",
    "returnMsg" : f"[Send] message send at {timestamp}.\n"
}
```

If User1 sends a message to User2, the response User2 gets will be in SMTP format shown below. And the message display on user 2 indicate the message received

```
151 def createMailMessage(sender, receiver, messageType, msg):
152     sendMsg = {
153         "header" : {
154             "To": receiver,
155             "From": sender,
156             "subject": messageType
157         },
158         "returnMsg": f"[Recv] {msg}"
159     }
160     return sendMsg
```

Message Prefix:

Prefix	Application Layer Message Format	StatusCode	Meaning
[Error]	HTTP	400	Invalid input or request
[Blocked]	HTTP	400	The user have fail too much and the username is blocked
No Prefix	HTTP	200	Success request
[Recv]	SMTP		Message receive for the sender
[Send]	HTTP	200	Message sent successfully to the receiver

Design Tradeoff:

- The group name exists forever once it is created. Even if all the users have deactivated (i.e /logout), the group name is still considered to exist or used
- One user can have multiple username
- No more valid user can be added to the program once the server started
- Users can create a group that doesn't contain themselves or can not add member to the group

Improvement

1. The group name can be created by another user if all the users within the group are deactivated. Method: When /creategroup is called, the server can first check if there are any active users in the group, if none are active users, then the group is valid for creation.
- One user can have multiple usernames: For improvement, the program should first check the credential.txt and make sure that one user can only have one password at the start of their server program.
 - Update the validUserList in a time period to account for the dynamic change of user.
 - Automatically add the creator the the group and having another command where the creator will able to add new member to the group

Issues:

- For /p2pvideo, the file in the UDP server can not open to view the video.

Reference:

Get idea from the sample code multi-thread client and server from

<https://webcms3.cse.unsw.edu.au/COMP3331/23T3/resources/90204>